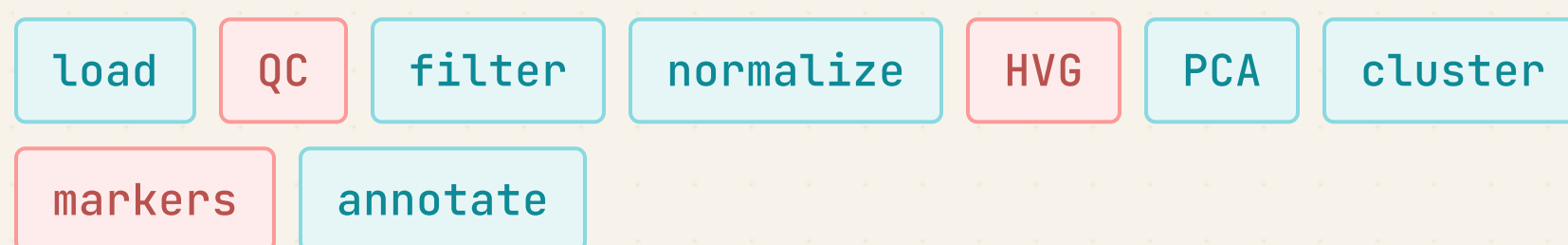


The Single-Cell *Cheat Sheet*

The entire Scanpy workflow — from raw counts to labelled cell types — in ten slides. Brain dataset throughout.



Swipe through the pipeline →

First, meet *AnnData*

.X

Expression matrix: cells × genes.
Its *meaning* changes as you process.

.obs

Per-**cell** table: QC, clusters, sample, cell type.

.var

Per-**gene** table: names, highly-variable flag.

.layers

Extra matrices, e.g. raw counts kept aside.

.obsm

Embeddings: X_pca, X_umap — what you plot.

.uns

Results & settings: marker tests, colors, params.

One object holds everything. Each step you run writes its result back into this same adata.

Read in, then *measure quality*

```
import scanpy as sc

adata = sc.read_10x_mtx("cortex/filtered/")
adata.var_names_make_unique()

# flag mitochondrial genes (human: MT-)
adata.var["mt"] = adata.var_names.str.startswith('MT-')
sc.pp.calculate_qc_metrics(adata,
                           qc_vars=["mt"], inplace=True)
```

Always print adata after loading. Then plot QC with `sc.pl.violin` before you cut anything.

Drop empty & *dying cells*

```
sc.pp.filter_cells(adata, min_genes=200)
sc.pp.filter_genes(adata, min_cells=3)

adata = adata[adata.obs.n_genes_by_counts < 6000]
adata = adata[adata.obs.pct_counts_mt < 10]
```

These numbers aren't universal. 6000 genes / 10% mito are starting points — read your *own* violin plots first.

Normalize, *then log*

```
# save raw counts BEFORE you change X
adata.layers["counts"] = adata.X.copy()

sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)
adata.raw = adata    # freeze for DE & plots
```

Why both? `normalize_total` removes depth differences between cells; `log1p` tames a handful of sky-high genes.

Keep signal, *cut noise*

```
sc.pp.highly_variable_genes(adata,  
                             n_top_genes=2000)  
adata = adata[:, adata.var.highly_variable]  
  
sc.pp.scale(adata, max_value=10)  
sc.tl.pca(adata)  
sc.pl.pca_variance_ratio(adata, n_pcs=50)
```

~2000 genes carry the biology. Use the variance plot's elbow to pick n_pcs for the next step (often 20-40).

UMAP, then *Leiden*

```
sc.pp.neighbors(adata, n_neighbors=15, n_pcs=30)
sc.tl.umap(adata)

sc.tl.leiden(adata, resolution=1.0)
sc.pl.umap(adata, color="leiden")
```

resolution is the dial: higher = more, smaller clusters. And remember — UMAP distances are for *looking*, not measuring.

Name your *cell types*

```
sc.tl.rank_genes_groups(adata, "leiden",  
                        method="wilcoxon")  
  
markers = {"Excitatory": ["SLC17A7"],  
          "Inhibitory": ["GAD1", "GAD2"],  
          "Astrocyte": ["AQP4", "GFAP"],  
          "Microglia": ["P2RY12"]}  
  
sc.pl.dotplot(adata, markers, "leiden")
```

Match markers → rename. Map cluster numbers to types and store as `adata.obs["cell_type"]`.

3 things that *trip everyone up*

1 • **.X** changes meaning

Raw counts → log-normalized → scaled, step by step. Save counts **before** you normalize.

2 • Slicing makes a *view*

Subsetting returns a view, not a copy. Add **.copy()** before you edit a subset.

3 • DE on the wrong matrix

Never run markers on scaled data. Use **adata.raw** (log-normalized) instead.

The whole pipeline, *one line*

load > QC > filter > norm > HVG > PCA > cluster >
annotate

Save this for your next dataset — and *follow* for more single-cell & bioinformatics walk-throughs.

↑ Save

+ Follow

△ Repost